

CS 301

2024-2025 Fall

Project Final Report

Group 98

Group Member:

Hasan Kuşpınar - 30633

Contents

1	Problem Description	3
1.1	Overview	3
1.2	Decision Problem	3
1.3	Optimization Problem	3
1.4	Example Illustration	4
1.5	Real World Applications	5
1.6	Hardness of the Problem	6
1.6.1	Proving NP	6
1.6.2	Proving NP-Hard	6
1.6.3	Proving NP-Complete	6
2	Algorithm Description	7
2.1	Brute Force Algorithm	7
2.1.1	Overview	7
2.1.2	Pseudocode	7
2.2	Heuristic Algorithm	8
2.2.1	Overview	8
2.2.2	Pseudocode	8
3	Algorithm Analysis	10
3.1	Brute Force Algorithm	10
3.1.1	Correctness Analysis	10
3.1.2	Time Complexity Analysis	10
3.1.3	Space Complexity Analysis	11
3.2	Heuristic Algorithm	12
3.2.1	Correctness Analysis	12
3.2.2	Time Complexity Analysis	12
3.2.3	Space Complexity Analysis	13
4	Sample Generation	14
4.1	Explanation	14
4.2	Pseudocode	14
4.3	Python Code	14
5	Algorithm Implementation	15
5.1	Brute Force MaxIndependentSet Python Code	15
5.1.1	Initial Test Of the Brute Force Algorithm	16
5.2	IsIndependentSet Python Code	17
5.3	FindMaxIndependentSetHeuristic Python Code	17
5.3.1	Initial Test Of the Heuristic Algorithm	19
5.4	PlotGraph Python Code	20
5.5	Main Function For Brute Force Python Code	21
5.6	Main Function For Heuristic Python Code	21
5.7	Plots	22

6	Experimental Analysis of the Performance	23
6.1	50 Runs	23
6.2	100 Runs	25
6.3	Comparison with Theoretical Time Complexity	26
7	Experimental Analysis of the Quality	27
7.1	Success Rate	27
7.2	Runtime Comparison	28
7.3	Conclusion	28
8	Experimental Analysis of the Correctness of the Implementation	29
8.1	Test Design	29
8.2	Test Results	30
8.3	Analysis of Black-Box Testing	30
8.4	Conclusion	30
9	Discussion	31
9.1	Algorithm Performance and Quality	31
9.2	Correctness Testing and Defects	31
9.3	Consistency Between Theoretical and Experimental Results	31
9.4	Limitations and Future Improvements	32
9.5	Conclusion	32
10	References	33

1 Problem Description

Name	Independent Set Problem
Input	An undirected graph $G(V, E)$ with n nodes and edge set E ; a positive integer k where $k \leq n$
Question	Given a graph G and integer k , is there a subset of vertices of size k such that no two vertices in the subset are connected by an edge?

Table 1: Description of the Independent Set Problem

1.1 Overview

The Independent Set Problem is a well-known problem in computer science and graph theory. It involves finding the largest subset of vertices in a given graph such that no two vertices in the subset are adjacent, i.e., are not connected by edges. This subset of vertices is called an "independent set."

1.2 Decision Problem

In its decision problem form, the Independent Set Problem asks whether there exists an independent set of a given size k in a graph $G(V, E)$, where V is the set of vertices and E is the set of edges. The question can be formulated as: "IS THERE A SUBSET OF V OF SIZE k SUCH THAT NO TWO VERTICES IN THIS SUBSET ARE CONNECTED BY AN EDGE IN E ?"

1.3 Optimization Problem

The optimization version of the Independent Set Problem aims to find the maximum independent set, i.e., the largest subset of vertices in G with no edges between them. The objective is to maximize the number of vertices in this independent set.

1.4 Example Illustration

The figure below displays examples for the Independent Set Problem. In the first graph the blue vertices represent the largest independent set, in other words the largest set of vertices that are not connected via edges. In the second graph the set of vertices are not independent since 3 of them are connected to other vertices via edges. The third one displays an independent set of vertices but it is not the largest set since the largest independent set included 3 vertices.

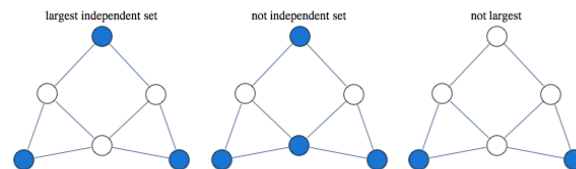


Figure 1: Example of an Independent and a not Independent Set in a Graph- source

In the figure below the relation between a Clique and an independent set can be seen. A clique is a subset of vertices in a graph such that each vertex in the set is adjacent to each other. Whereas an independent set is the subset of vertices that are not adjacent to each other; Therefore, taking an independent set from the complimentary of the Independent graph would result in a Clique.

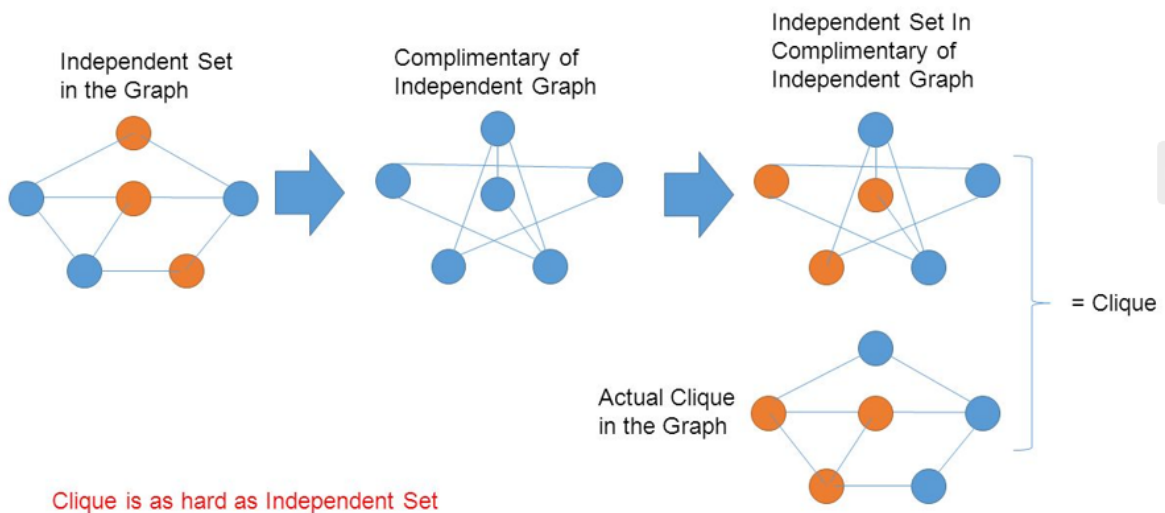


Figure 2: Reducing an Independent Set to Clique-source

1.5 Real World Applications

The Independent Set Problem has various applications in the real life including:

- **Matching Molecular Structures by Clique Detection:** In this approach, each molecule is represented as a graph, with vertices as atoms and edges as bonds or spatial relationships. To find a structural match, researchers build a "correspondence graph" where vertices represent possible atom pairings across the two molecules, and edges connect pairs that spatially or chemically match. By detecting cliques (fully connected subgraphs) within this correspondence graph, researchers can identify the maximum common subgraph (MCS), which represents the largest shared structure between the molecules. The Independent Set Problem is relevant here as it helps filter out incompatible pairings by identifying non-interfering sets of atoms, aiding in stable structure identification. This technique is particularly useful in pharmacophore mapping and drug design, where structural compatibility is key.¹
- **Wireless Communication:** In wireless communication networks, independent sets are used to organize large-scale sensor networks by grouping nodes effectively such that direct communication between each other is minimized. An independent set in a communication graph consists of nodes that cannot directly communicate with each other. This property is particularly useful in clustering strategies, as it makes it possible to create clusters of nodes that operate without interference, allowing a hierarchical organization of the network. By selecting independent sets, the network can designate certain nodes to control clusters, which reduces network complexity and helps manage data transmission efficiently.²
- **Covering Location:** The concept of independent sets is relevant regarding "covering location problems" especially in cases involving undesirable facility placement. In these problems, the main goal is often to locate facilities in a way that they are both isolated from each other and from certain areas, such as residential zones, to minimize negative impacts like pollution or noise. The Independent Set Problem can be used for these since the independent sets are not adjacent, i.e., has no direct connection and also the independent sets help reduce the coverage.³
- **Scheduling:** The Independent Set Problem can also be used to efficiently organize and resolve conflicts in imaging activities for large satellite constellations. The approach models the satellite scheduling problem as a graph, where vertices represent potential imaging tasks and edges indicate tasks that cannot be performed simultaneously. In this framework, an optimal schedule corresponds to a maximal independent set of vertices in the graph.⁴

¹Eleanor J Gardiner, Peter J Artymiuk, and Peter Willett, "Clique-Detection Algorithms for Matching Three-Dimensional Molecular Structures," *Journal of Molecular Graphics and Modelling* 15, no. 4 (August 1997): 245–53, [https://doi.org/10.1016/s1093-3263\(97\)00089-2](https://doi.org/10.1016/s1093-3263(97)00089-2).

²Tim Nieberg. Independent and dominating sets in wireless communication graphs. *IEEE Transactions on Software Engineering - TSE* (2006).

³Mostafa Khatami and Amir Salehipour, "The Gradual Minimal Covering Location Problem," *SSRN Electronic Journal* (2020), <https://doi.org/10.2139/ssrn.3522777>.

⁴Duncan Eddy and Mykel J. Kochenderfer, "A Maximum Independent Set Method for Scheduling Earth-Observing Satellite Constellations," *Journal of Spacecraft and Rockets* 58, no. 5 (September 2021): 1416–29, <https://doi.org/10.2514/1.a34931>.

1.6 Hardness of the Problem

1.6.1 Proving NP

To show that the Independent Set Problem is in NP, we note that for a given set S , we can verify in polynomial time whether S is an independent set of the required size k . For any instance where a solution exists, there exists an independent set of size k . If no such set exists, the answer is clearly "no." Therefore, IS is in NP.

1.6.2 Proving NP-Hard

To establish NP-hardness, we reduce from the 3-SAT problem. Given a 3-SAT formula

$$F = C_1 \wedge C_2 \wedge \dots \wedge C_m$$

where each clause C_i consists of three literals, we construct a graph G as follows:

- For each literal in each clause, create a node labeled with the literal's variable.
- For each clause, add edges between nodes representing its literals, forming a "clause gadget."
- For each pair of nodes labeled x_i and $\neg x_i$, add an edge between them.

We then check if there is an independent set of size m in G . If such an independent set exists, it indicates a satisfying assignment for F ; otherwise, F is unsatisfiable.

Correctness of Reduction

- **If F is satisfiable:** A satisfying assignment can provide an independent set S by selecting one node per clause gadget corresponding to a satisfied literal.
- **If there exists an independent set of size m in G :** We can derive a satisfying assignment for F by setting the truth values based on the literals in S .

1.6.3 Proving NP-Complete

To prove that the Independent Set Problem is NP-complete, we need to verify its participation in both NP and NP-hard sets. Firstly, it has been proved that the Independent Set Problem is NP by showing that it is verified in polynomial time. After that we proved that the Independent Set Problem is NP-hard by reducing from the 3-SAT problem. Therefore, since the Independent Set Problem is both in NP and NP-hard, it is NP-complete.⁵

⁵Alexa Sharp, Proving a Problem is NP-complete, accessed November 11, 2024, <https://www.cs.cornell.edu/courses/cs482/2005su/handouts/NPComplete.pdf>.

2 Algorithm Description

2.1 Brute Force Algorithm

2.1.1 Overview

The algorithm implemented below for solving the independent set problem is a brute-force approach, which guarantees a correct solution but is computationally intensive due to its exponential time complexity.

2.1.2 Pseudocode

The below algorithm finds the maximum independent set in a given graph. It finds the maximum independent set by iterating over every possible subset of the graph to check if any two vertices in that subset has an edge and marking the current graph as the max if it is larger than the max.

Algorithm 1 Find Maximum Independent Set

```
1: Function FindMaxIndependentSet(graph)
2: Initialize max_independent_set as an empty list
3: Initialize subsets as a list containing the empty subset
4: Let vertices be the list of vertices in graph
5: for each vertex in vertices do
6:   Initialize temp_subsets as an empty list
7:   for each subset in subsets do
8:     Create temp_subset = subsets  $\cup$  {vertex}
9:     Add temp_subset to temp_subsets
10:  end for
11:  Add all temp_subsets to subsets
12: end for
13: for each subset in subsets do
14:   if is_independent_set(graph, subset) and size of subset > size of
      max_independent_set then
15:     Update max_independent_set = subset
16:   end if
17: end for
18: return max_independent_set
```

In the algorithm above, first for loop creates every possible subset by iterating over every vertex in the graph and creating a new subset after adding the vertex in each iteration. Moreover, the second for loop iterates over the subsets to check each pair of vertices in them to see if they have an edge between them using the `IsIndependentSet` function. After that, it checks whether there is an edge and the size of the current subset is bigger than the max. If so it updates the max as the current. Lastly, when the for loop finishes it returns the max.

Algorithm 2 Check if a Set is an Independent Set

```
1: Function IsIndependentSet(graph, subset)
2: for each  $i$  from 0 to  $\text{len}(\text{subset}) - 1$  do
3:   for each  $j$  from  $i + 1$  to  $\text{len}(\text{subset}) - 1$  do
4:     if there exists an edge between the vertices( $i, j$ ) then
5:       return False
6:     end if
7:   end for
8: end for
9: return True
```

The above algorithm is for the function to check if the given subset is independent. It iterates over each vertex pair to check if they have an edge between them. If so it returns false. If no edge is found it returns true.

2.2 Heuristic Algorithm

2.2.1 Overview

The following pseudocode describes an algorithm for finding a maximal independent set in a given graph with a heuristic approach which does not guarantee finding a correct solution but it guarantees speed.

2.2.2 Pseudocode

The algorithm below is designed to find the maximum independent set in a given graph. It starts by iterating over each vertex in the graph and choosing an initial one to try adding its neighbors to the set that would consist of the independent vertices. The neighbor vertex is included in the set if it does not make it unfeasible. The algorithm then updates the graph's adjacency matrix by removing all edges connected to the vertices in the solution and their neighbors. It searches for the vertex with the highest degree and tries to add its neighbor in a similar way. This procedure is repeated until all vertices have a degree of zero.

After finding an initial solution, it tries to find another vertex that does not make the set unfeasible after adding it to it. If there is no such vertex, the algorithm attempts to create a feasible vertex for inclusion by adjusting the vertices already in the solution. Finally, it updates the independent set if the current one is larger, then it returns that set after the outer iteration is over. ⁶

⁶1. Onur Ugurlu, "A New Heuristic Algorithm to Solve the Maximum Independent Set Problem," *Mathematical and Computational Applications* 18, no. 3 (December 1, 2013): 495–501, <https://doi.org/10.3390/mca18030495>.

Algorithm 3 Heuristic Algorithm for Maximum Independent Set

```
1: Function FindMaxIndependentSetHeuristic(graph)
2: Initialize  $I^* \leftarrow \emptyset$  (empty independent set)
3: for each vertex  $v_i \in V$  do
4:   Update the adjacency matrix of  $G$ 
5:   Add neighbors of  $v_i$  to  $I_i$ , if they do not make  $I_i$  unfeasible
6:   repeat
7:     Find the vertex  $v_m$  with the maximum degree
8:     Add neighbors of  $v_m$  to  $I_i$ , if they do not make  $I_i$  unfeasible
9:     Delete all edges adjacent to vertices in  $I_i$  and their neighbors
10:  until  $E = \emptyset$  (No edges remain in the graph)
11:  for each vertex  $v_k \in V$  do
12:    if  $v_k \notin I_i$  and all vertices in  $N(v_k) \notin I_i$  then
13:      Add  $v_k$  to  $I_i$ 
14:    end if
15:    if  $v_m \notin I_i$  and only one neighbor  $v_n \in I_i$  then
16:      Remove  $v_n$  from  $I_i$ 
17:      Add  $v_m$  to  $I_i$ 
18:      Go to Step 16
19:    end if
20:  end for
21:  if  $|I_i| > |I^*|$  then
22:    Update  $I^* \leftarrow I_i$  and  $\alpha \leftarrow |I_i|$ 
23:  end if
24: end for
25: Return:  $I^*$ 
```

3 Algorithm Analysis

3.1 Brute Force Algorithm

3.1.1 Correctness Analysis

The correctness of the maximum independent set algorithm (MAXINDEPENDENTSET) can be stated as follows:

Theorem 1. *The MAXINDEPENDENTSET algorithm returns the largest independent set of the input graph, containing the maximum number of vertices such that no two vertices in the set are adjacent.*

Proof. The algorithm iterates through all possible subsets of vertices in the graph, checking each subset for independence using the ISINDEPENDENTSET method. If a subset is independent and its size is greater than the currently recorded maximum independent set, the algorithm updates the maximum independent set with this subset, which ensures the final result would be the set with the maximum size. Since the algorithm explores every possible subset, it ensures that every potential independent set is considered. Therefore, the algorithm returns the largest independent set of the input graph, containing the maximum number of vertices such that no two vertices in the set are adjacent. $\square$

3.1.2 Time Complexity Analysis

For a graph $G(V, E)$, where $|V|$ is the number of vertices and $|E|$ is the number of edges in the graph, to find the time complexity of the algorithm we need to check the following:

- **Subset Generation:** Takes at most $O(2^{|V|} \cdot |V|)$
- **Checking Independent Sets:** Takes at most $O(2^{|V|} \cdot |V|^2)$
- **Updating Max Set:** Takes at most $O(2^{|V|} \cdot |V|)$

Therefore, the worst case time complexity is:

$$\mathcal{O}(2^{|V|} \cdot |V|^2)$$

3.1.3 Space Complexity Analysis

The space complexity of the algorithm could be found by looking at the data structures that are used to store the sets and subsets used during the process.

- **The list of nodes:** We use a list to store the nodes(vertices) which requires $O(|V|)$ space.
- **The maximum independent set list:** We use a list to store the possible maximum independent set which requires $O(|V|)$ space since it can be equal to the original graph.
- **All possible subsets:** As the first step in the algorithm, we create all possible subsets from the original graph. Since there are $|V|$ nodes, the number of subsets is $2^{|V|}$. And since the number of nodes(vertices) in a subset can go up to $|V|$, the space required for this operation is $O(2^{|V|} \cdot |V|)$.
- **Temporary new subsets:** In each iteration over the nodes(vertices) we create a list to store the possible new subsets and append a subset to it in each iteration over the original subsets. Since the number of nodes in a subset can go up to $|V|$ and there are $2^{|V|}$ subsets, this requires $O(2^{|V|} \cdot |V|)$ space.

Therefore, the space complexity for this algorithm is:

$$\mathcal{O}(2^{|V|} \cdot |V|)$$

3.2 Heuristic Algorithm

3.2.1 Correctness Analysis

The correctness of the maximum independent set heuristic algorithm can be stated as follows:

Theorem 1. *The MAXINDEPENDENTSETHEURISTIC algorithm returns a feasible independent set of the input graph, which is not necessarily the largest but satisfies the independence property.*

Proof. The algorithm iteratively constructs an independent set by starting with a single vertex and incrementally adding vertices that do not violate the independence property. At every step:

- It checks if adding a vertex maintains the independence property (no two vertices in the set are adjacent).
- It removes the vertices and their neighbors from the graph to ensure feasibility in subsequent iterations.

Since the algorithm strictly adheres to the independence property at every step, the resulting set is guaranteed to be independent. However, as it chooses vertices based on degree, the solution is not guaranteed to be optimal or the largest independent set. $\square$

3.2.2 Time Complexity Analysis

For a graph $G(V, E)$, where $|V|$ is the number of vertices and $|E|$ is the number of edges in the graph, the time complexity can be broken down as follows:

- **Outer Iteration:** The algorithm iterates over each vertex in the graph, which takes $O(|V|)$ in total. And all the below processes take place inside of this outer loop.
- **Neighbor Selection:** For each vertex in the graph, we iterate over its neighbors to check independence, which takes $O(|E|)$ in total since all edges are examined.
- **Copying:** The algorithm copies the entire graph for once in each iteration of the outer loop, which takes $O(|V| + |E|)$ in total.
- **While Loop:** Inside the while loop, the graph is updated by finding the vertex with the maximum degree. Computing the maximum degree involves iterating over the vertices, which takes $O(|V|)$ per iteration. In the worst case, the loop runs $O(|V|)$ times, resulting in $O(|V|^2)$ work for degree calculations.
- **Updating the Graph:** Removing vertices and their neighbors requires $O(|V| + |E|)$ in total across all steps.

The dominant term is the while loop with $O(|V|^2)$ time complexity and that would be repeated for $|V|$ times due to the outer loop. Therefore, the worst-case time complexity is:

$$\mathcal{O}(|V|^3)$$

3.2.3 Space Complexity Analysis

The space complexity of the algorithm can be analyzed based on the data structures used:

- **Graph Representation:** The graph is stored in a networkx graph which stores it as an adjacency list, requiring $O(|V| + |E|)$ space.
- **Temporary Graph Copy:** A temporary graph copy is made at each step, which also requires the same amount of space with the first step, that is $O(|V| + |E|)$ space.
- **Independent Set Storage:** The independent set is stored as a list, which requires $O(|V|)$ space.
- **Neighbor Tracking:** A temporary list of neighbors is created at each iteration, requiring $O(|V|)$ space.

Therefore, the space complexity for this algorithm is:

$$\mathcal{O}(|V| + |E|)$$

4 Sample Generation

4.1 Explanation

The sample generation for the Maximum Independent Set Problem requires creating a graph with the given parameters, **number of vertices** and the threshold for inserting edges **edge probability**. This allows for the creation of various graphs for testing the algorithm.

4.2 Pseudocode

Algorithm 4 Generate Random Graph

```
1: Function GenerateRandomGraph(num_vertices, edge_probability)
2: Initialize an empty graph  $G$ 
3: Add vertices  $0, 1, \dots, \text{num\_vertices} - 1$  to  $G$ 
4: for each pair of vertices  $(i, j)$  with  $0 \leq i < j < \text{num\_vertices}$  do
5:   Generate a random number  $r$  in the range  $[0, 1]$ 
6:   if  $r \leq \text{edge\_probability}$  then
7:     Add edge  $(i, j)$  to  $G$ 
8:   end if
9: end for
10: return  $G$ 
```

The above algorithm generates graphs for the given parameters **num_vertices** and **edge_probability**. It adds vertices according to the given **num_vertices** parameter to the graph. Then, for each pair of vertices it adds edges between them until the random number generated passes the threshold **edge_probability**. Lastly, it returns the generated graph.

4.3 Python Code

```
1 def generate_random_graph(num_vertices, edge_probability):
2     #create the graph object
3     G = nx.Graph()
4     #add vertices into the graph according to the num_vertices
5     G.add_nodes_from(range(num_vertices))
6
7     #iterate over each vertex pair
8     for i in range(num_vertices):
9         for j in range(i + 1, num_vertices):
10            #if random number is not bigger than edge_probability
11            if random.random() < edge_probability:
12                #insert an edge
13                G.add_edge(i, j)
14
15     return G
```

Listing 1: Function to generate a random graph

5 Algorithm Implementation

5.1 Brute Force MaxIndependentSet Python Code

The Brute Force MaxIndependentSet code has been tried with 20 different num_vertices and edge_probability pairs. For each test; number of vertices, number of edges, edge probability, total time and the size of the independent set is recorded in the name of the file that the graph plot is saved to.

```
1 def find_max_independent_set(graph):
2     #for recording the time
3     start_time = time.time ()
4     max_independent_set = []
5
6     #initialize a subset list containing the empty subset
7     subsets = [[]]
8
9     #create a list to store vertices from the graph
10    vertices = list(graph.nodes())
11
12    #generate all subsets by adding each vertex to existing subsets
13    for vertex in vertices:
14        temp_subsets = []
15
16        #add the current vertex to each existing subset to create new
subsets
17        for subset in subsets:
18            temp_subset = subset + [vertex]
19            temp_subsets.append(temp_subset)
20
21        #add new subsets to the list of all subsets
22        subsets.extend(temp_subsets)
23
24    #iterating over each subset ot find the max
25    for subset in subsets:
26        #check if the subset is an independent set and the len of the
subset is bigger than the max
27        if is_independent_set(graph,subset) and len(subset) > len(
max_independent_set):
28            max_independent_set = subset
29
30    #calculating the total time taken by the algorithm
31    end_time = time.time ()
32    total_time = end_time - start_time
33
34    return max_independent_set, total_time
```

Listing 2: Function to find the maximum independent set in a graph

5.1.1 Initial Test Of the Brute Force Algorithm

To see the effect of the exponential complexity, one can compare the graph instances with $|V| = 5$ and $|V| = 15$. The instance with $|V| = 5$ has a time of **0.0000001** seconds and the instance with $|V| = 15$ has a time of **0.082098** seconds. It can be observed that even though the $|V|$ is thrice its previous value, the time is 800 thousand times more its previous value. In the table below; $|V|$ represents the number of vertices, $|E|$ represents the number of edges, p represents the edge probability, $|S|$ represents the maximum independent set size and t represents the time.

Graph Instance	$ V $	$ E $	p	$ S $	t (s)
1	5	3	0.5	3	0.0000001
2	6	8	0.5	3	0.0000001
3	7	10	0.5	4	0.0000001
4	8	10	0.5	4	0.0005000
5	9	18	0.5	4	0.0010130
6	10	10	0.3	5	0.0010230
7	10	14	0.4	5	0.0010000
8	10	22	0.5	4	0.0009970
9	10	25	0.6	4	0.0010010
10	11	27	0.5	4	0.0020000
11	11	28	0.4	4	0.0010190
12	12	18	0.3	7	0.0060130
13	12	38	0.5	4	0.0041030
14	12	43	0.6	4	0.0040150
15	13	33	0.4	5	0.0090010
16	13	42	0.5	4	0.0070610
17	14	49	0.5	4	0.0156890
18	15	46	0.5	4	0.0820980
19	16	56	0.5	5	0.0686230
20	16	77	0.7	4	0.0686620

Table 2: Initial Test Results

5.2 IsIndependentSet Python Code

```
1 def is_independent_set(graph, subset):
2     #iterating over each pair of vertices
3     for i in range(len(subset)):
4         for j in range(i + 1, len(subset)):
5             # Check if there's an edge between any two nodes in the
6             subset
7             if graph.has_edge(subset[i], subset[j]):
8                 return False
9     return True
```

Listing 3: Function to check if a given subset is independent

5.3 FindMaxIndependentSetHeuristic Python Code

The FindMaxIndependentSetHeuristic code has been tried with 20 different num_vertices and edge_probability pairs just like the brute force one. For each test; number of vertices, number of edges, edge probability, total time and the size of the independent set is recorded in the name of the file that the graph plot is saved to.

```
1 def find_max_independent_set(graph):
2     best_independent_set = set() #stores the largest independent set
3     #found
4     vertices = list(graph.nodes()) #list of vertices in the graph
5     #for recording the time
6     start_time = time.time ()
7
8     for initial_vertex in vertices:
9         independent_set = set() #independent set for the current
10        iteration
11        independent_set.add(initial_vertex)
12
13        #add neighbors of the initial vertex to the independent set if
14        #feasible
15        neighbors = list(graph.neighbors(initial_vertex))
16        for neighbor in neighbors:
17            if all(not graph.has_edge(neighbor, node) for node in
18            independent_set):
19                independent_set.add(neighbor)
20
21        #update adjacency matrix by removing edges adjacent to nodes in
22        #the independent set and their neighbors
23        temp_graph = graph.copy()
24        nodes_to_remove = set(independent_set)
25        for node in independent_set:
26            if node in temp_graph:
27                nodes_to_remove.update(temp_graph.neighbors(node))
28        temp_graph.remove_nodes_from(nodes_to_remove)
29
30        #repeat the improvement process
31        while temp_graph.number_of_edges() > 0:
32            #find the vertex with the maximum degree
33            max_degree_vertex = max(temp_graph.degree, key=lambda x: x
34            [1])[0]
35
36            #add neighbors of the maximum degree vertex if feasible
```

```

31         neighbors = list(temp_graph.neighbors(max_degree_vertex))
32         for neighbor in neighbors:
33             if all(not graph.has_edge(neighbor, node) for node in
independent_set):
34                 independent_set.add(neighbor)
35
36         #update the adjacency matrix
37         nodes_to_remove = set(independent_set)
38         for node in independent_set:
39             if node in temp_graph:
40                 nodes_to_remove.update(temp_graph.neighbors(node))
41         temp_graph.remove_nodes_from(nodes_to_remove)
42
43         #try to add remaining vertices to the independent set
44         for vertex in graph.nodes():
45             if vertex not in independent_set and all(
46                 neighbor not in independent_set for neighbor in graph.
neighbors(vertex)
47             ):
48                 independent_set.add(vertex)
49
50         #try shifting vertices to create feasible additions
51         for vertex in vertices:
52             if vertex not in independent_set:
53                 neighbors = list(graph.neighbors(vertex))
54                 for neighbor in neighbors:
55                     if neighbor in independent_set and len(neighbors)
== 1:
56                     independent_set.remove(neighbor)
57                     independent_set.add(vertex)
58                     break
59
60         #update the best independent set if the current one is larger
61         if len(independent_set) > len(best_independent_set):
62             best_independent_set = independent_set
63
64         #calculating the total time taken by the algorithm
65         end_time = time.time ()
66         total_time = end_time - start_time
67
68         return best_independent_set, total_time

```

Listing 4: Function to find a maximal independent set with heuristic approach

5.3.1 Initial Test Of the Heuristic Algorithm

Unlike the brute force one, the heuristic one's times seems to not go up rapidly. It has a more feasible time distribution throughout the graph instances with a polynomial complexity as desired. In the table below; $|V|$ represents the number of vertices, $|E|$ represents the number of edges, p represents the edge probability, $|S|$ represents the maximum independent set size and t represents the time.

Graph Instance	$ V $	$ E $	p	$ S $	t (s)
1	5	4	0.5	3	0.0009982
2	6	8	0.5	3	0.0010492
3	7	15	0.5	4	0.0000001
4	8	12	0.5	4	0.0009744
5	9	17	0.5	4	0.0010042
6	10	13	0.3	5	0.0009994
7	10	20	0.4	5	0.0011162
8	10	22	0.5	4	0.0010535
9	10	22	0.6	4	0.0010857
10	11	19	0.5	4	0.0019907
11	11	33	0.4	4	0.0010116
12	12	14	0.3	7	0.0020027
13	12	37	0.5	4	0.0020151
14	12	37	0.6	4	0.0020115
15	13	32	0.4	5	0.0022749
16	13	41	0.5	4	0.0020711
17	14	44	0.5	4	0.0019700
18	15	46	0.5	4	0.0040881
19	16	61	0.5	5	0.0040175
20	16	79	0.7	4	0.0030171

Table 3: Initial Test Results for Heuristic Algorithm

5.4 PlotGraph Python Code

The PlotGraph function plots the given graph while highlighting the maximum independent set in it. The function also saves the graph plot to the same directory with the project. The number of vertices, number of edges, edge probability, total time and the size of the independent set are written into the name of the saved file to be further used in the results.

```
1 def plot_graph_with_independent_set(G, independent_set,
2   edge_probability, total_time):
3     pos = nx.spring_layout(G)
4
5     filename = f" graph_ {len(G.nodes())}_{len(G.edges())}_{
6   edge_probability}_{total_time:.6f}_{len(independent_set)}.png"
7     # Draw vertices with different colors for the independent set and
8   other vertices
9     vertex_colors = ['lightblue' if vertex not in independent_set else
10    'orange' for vertex in G.nodes]
11
12    # Draw the graph
13    nx.draw(G, pos, with_labels=True, node_color=vertex_colors,
14    node_size=500, font_size=10, font_color='black', edge_color='gray')
15
16    # Highlight vertices in the independent set
17    nx.draw_networkx_nodes(G, pos, nodelist=independent_set, node_color
18    ='orange')
```

```
13 plt.title("Graph with Maximum Independent Set Highlighted")
14 plt.savefig(filename)
15
16 plt.show()
```

Listing 5: Function to plot a given graph

5.5 Main Function For Brute Force Python Code

The main method for brute force algorithm iterates for 20 times, while it iterates it requests the user to input both `num_vertices` and `edge_probability` parameters. After it takes the inputs it generates the graph, finds the maximum independent set, plots the graph and saves it. This is repeated in each iteration leading to 20 different input pairs being tested.

```
1 def main():
2     for i in range(20):
3         num_vertices = int(input(f"Enter the number of vertices for
graph {i+1}: "))
4         edge_probability = float(input(f"Enter the edge probability for
graph {i+1} (between 0 and 1): "))
5
6         G = generate_random_graph(num_vertices, edge_probability)
7         print("Generated Graph Edges:", list(G.edges()))
8
9         max_independent_set, total_time = find_max_independent_set(G)
10        print("Maximum Independent Set:", max_independent_set)
11        print("Size of Maximum Independent Set:", len(
max_independent_set))
12
13        plot_graph_with_independent_set(G, max_independent_set,
edge_probability, total_time)
```

Listing 6: The function for the brute force main method

5.6 Main Function For Heuristic Python Code

The main method for heuristic algorithm iterates for 20 times, while it iterates it requests the user to input both `num_vertices` and `edge_probability` parameters, just like the brute force one. After it takes the inputs it generates the graph, finds the maximum independent set, plots the graph and saves it. This is repeated in each iteration leading to 20 different input pairs being tested.

```
1 def main():
2     for i in range(20):
3         num_vertices = int(input(f"Enter the number of vertices for
graph {i+1}: "))
4         edge_probability = float(input(f"Enter the edge probability for
graph {i+1} (between 0 and 1): "))
5
6         G = generate_random_graph(num_vertices, edge_probability)
7         print("Generated Graph Edges:", list(G.edges()))
8
9         max_independent_set, total_time = find_max_independent_set(G)
10        print("Maximum Independent Set:", max_independent_set)
11        print("Size of Maximum Independent Set:", len(
max_independent_set))
12
13        plot_graph_with_independent_set(G, max_independent_set,
edge_probability, total_time)
```

Listing 7: The function for the heuristic main method

5.7 Plots

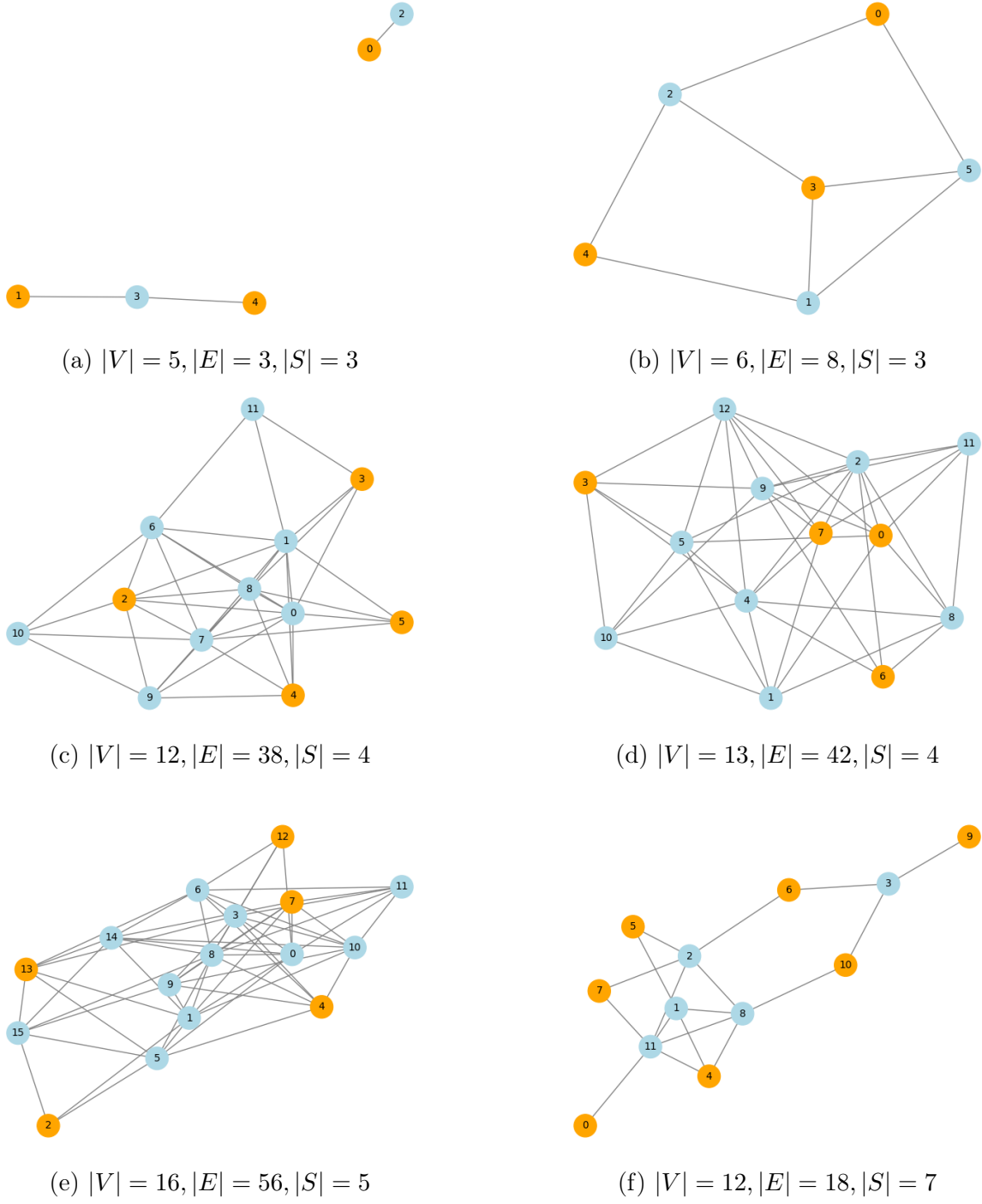


Figure 3: Examples of different plots

6 Experimental Analysis of the Performance

6.1 50 Runs

The performance of the heuristic algorithm was analyzed over 50 runs on various graph sizes. The results are presented in Table 4 below:

Graph Size	Mean Runtime	Std. Dev.	C.I. 90%	C.I. 95%
10	0.0009	0.0005	(0.0008, 0.0010)	(0.0007, 0.0010)
20	0.0041	0.0008	(0.0039, 0.0043)	(0.0039, 0.0043)
40	0.0236	0.0032	(0.0228, 0.0243)	(0.0227, 0.0245)
80	0.1605	0.0151	(0.1569, 0.1640)	(0.1562, 0.1647)
160	1.1672	0.0523	(1.1548, 1.1797)	(1.1524, 1.1821)
320	9.2005	0.4311	(9.0980, 9.3029)	(9.0779, 9.3230)

Table 4: Performance analysis of the heuristic algorithm with 50 runs

The table presents the following metrics:

- **Mean Runtime:** The average runtime over 50 runs for each graph size.
- **Standard Deviation (Std. Dev.):** The variation in runtime throughout the runs.
- **Confidence Intervals (C.I.):** The range in which the true mean running time lies with a specified confidence level:
 - **90% C.I.:** This indicates that, the true mean running time is within this range with 90% probability.
 - **95% C.I.:** Similarly, the true mean running time is within this range with 95% probability.

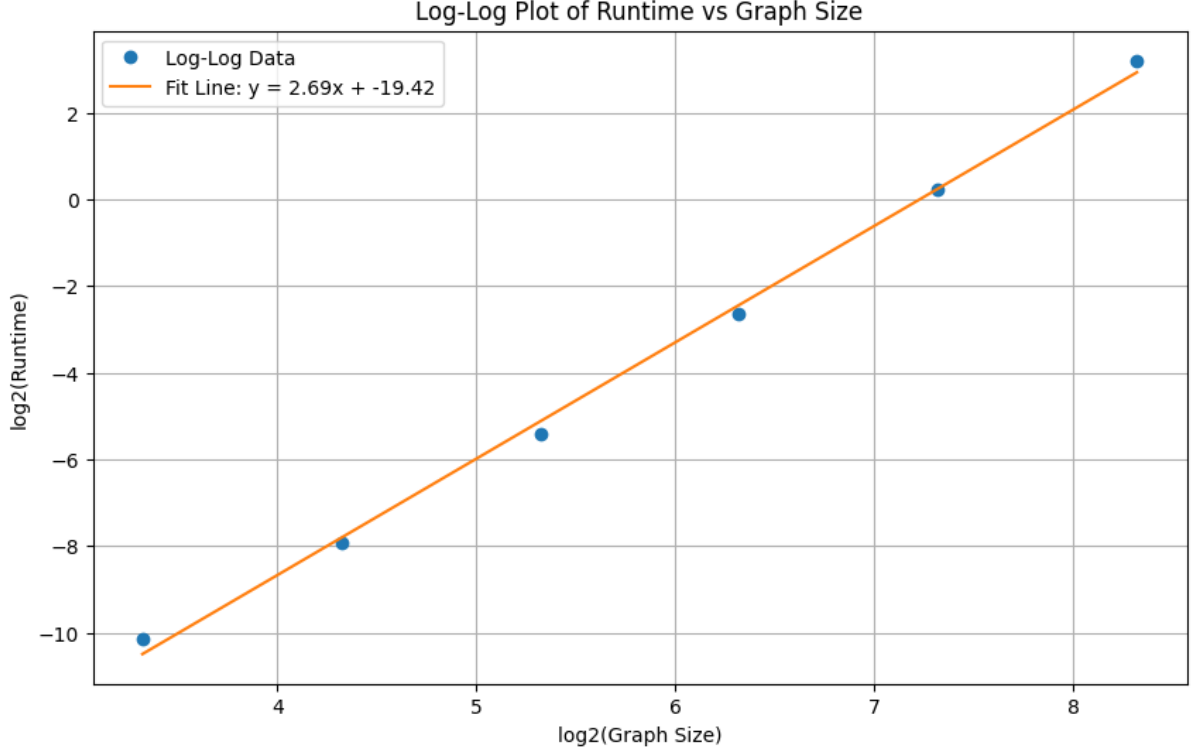


Figure 4: Log-log plot of the heuristic algorithm for 50 runs

Figure 4 shows the log-log plot of the algorithm's runtime over 50 runs. The fitted line has the equation:

$$\log_2(T(n)) = 2.69 \cdot \log_2(n) - 19.42$$

Converting this to the original scale, the time complexity is expressed as:

$$T(n) = 2^{-19.42} \cdot n^{2.69}$$

Thus, the time complexity derived from the log-log plot for 50 runs is:

$$\mathcal{O}(n^{2.69})$$

6.2 100 Runs

The heuristic algorithm was further analyzed over 100 runs on the same graph sizes. The results are presented in Table 5 below:

Graph Size	Mean Runtime	Std. Dev.	C.I. 90%	C.I. 95%
10	0.0009	0.0005	(0.0008, 0.0010)	(0.0008, 0.0010)
20	0.0048	0.0048	(0.0040, 0.0056)	(0.0039, 0.0058)
40	0.0227	0.0031	(0.0222, 0.0232)	(0.0221, 0.0233)
80	0.1592	0.0250	(0.1550, 0.1633)	(0.1542, 0.1642)
160	1.3236	0.2404	(1.2837, 1.3635)	(1.2759, 1.3713)
320	12.3305	1.0530	(12.1557, 12.5053)	(12.1216, 12.5394)

Table 5: Performance analysis of the heuristic algorithm with 100 runs

Similar to the analysis for 50 runs, the table presents the mean runtime, standard deviation, and confidence intervals. Due to the larger sample size, the mean and standard deviation values and the confidence intervals for the mean running times has increased reliability.

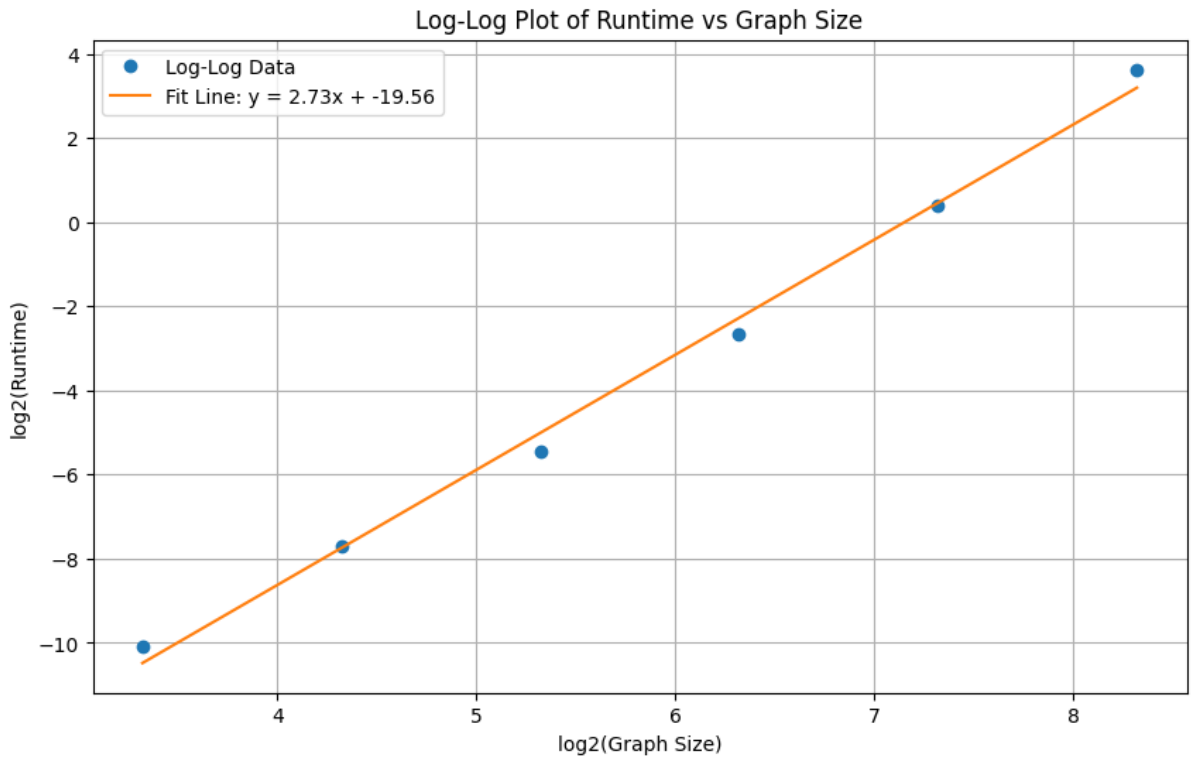


Figure 5: Log-log plot of the heuristic algorithm for 100 runs

Figure 5 shows the log-log plot of the algorithm's runtime over 100 runs. The fitted line has the equation:

$$\log_2(T(n)) = 2.73 \cdot \log_2(n) - 19.56$$

Converting this to the original scale, the time complexity is expressed as:

$$T(n) = 2^{-19.56} \cdot n^{2.73}$$

Thus, the time complexity derived from the log-log plot for 100 runs is:

$$\mathcal{O}(n^{2.73})$$

6.3 Comparison with Theoretical Time Complexity

The theoretical analysis determined that the worst-case time complexity of the algorithm is $\mathcal{O}(n^3)$. The experimentally derived time complexities of $\mathcal{O}(n^{2.69})$ and $\mathcal{O}(n^{2.73})$ from the log-log plots are slightly better than the theoretical bound. This discrepancy arises because the experimental setup does not always reflect the worst-case behavior. For example:

- The graph structures used in the experiments might not always be the ones that lead to the worst case.
- The heuristic may perform more efficiently on sparse graphs or graphs with specific degree distributions, reducing the runtime.

Overall, the experimental results confirm that the algorithm runs in polynomial time and align closely with the theoretical expectations. However, the difference between two time complexities occur mainly due to the factors explained above and with the sample size growing and getting closer to the worst case the time gets closer to the theoretical $\mathcal{O}(n^3)$ time.

7 Experimental Analysis of the Quality

7.1 Success Rate

A test was designed to compare the heuristic algorithm with the brute force algorithm with 8 different graph sizes for 30 iterations, however these graph sizes were not bigger than 25 since doing so would require a very long time for the brute force algorithm. In each iteration a random edge probability was assigned leading to different edge probabilities being tested. By comparing the size of the max independent set in both algorithms for these different sets of values it is possible to examine the quality of the heuristic algorithm.

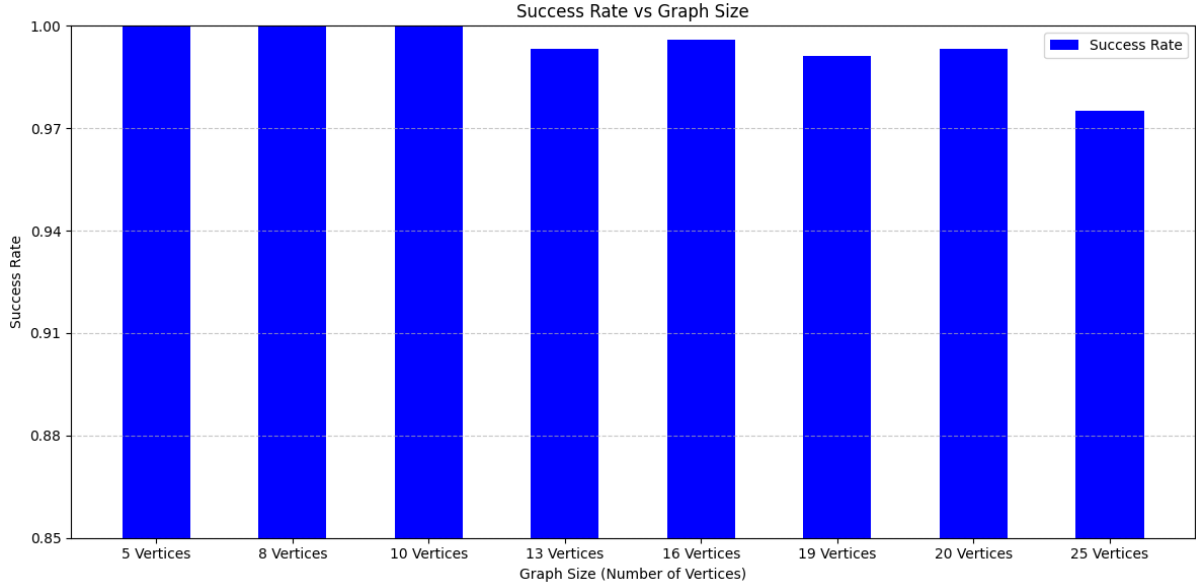


Figure 6: A plot showing the success rate of the heuristic algorithm with respect to graph size

After looking at the results of the test in figure 6, it can be concluded that for small sizes of the graphs the algorithm does not seem to make many mistakes leading to a success rate of at least 97%. However, even though it seems to have a high success rate, the decrease of the success rate with the increasing graph size could be seen from this test with small problem size as well. The graphs with 5, 8, 10 vertices have a success rate of 100%, after these the success rate drops down slightly until the graphs with 25 vertices which has a success rate of 97%.

The heuristic algorithm's quality diminishes slightly as the problem size increases, but the impact is minimal. The difference in success rates indicates that the heuristic is well-designed to balance computational efficiency and solution quality which can be as a result of it being slightly slower than some of the heuristics. For small problem sizes, the algorithm can successfully find optimal independent sets just like the brute force. For the larger problem sizes (such as 25), even though it starts to struggle, the quality is still very high which indicates that the algorithm can find close to optimal independent sets even for the larger problem sizes while being fast.

7.2 Runtime Comparison

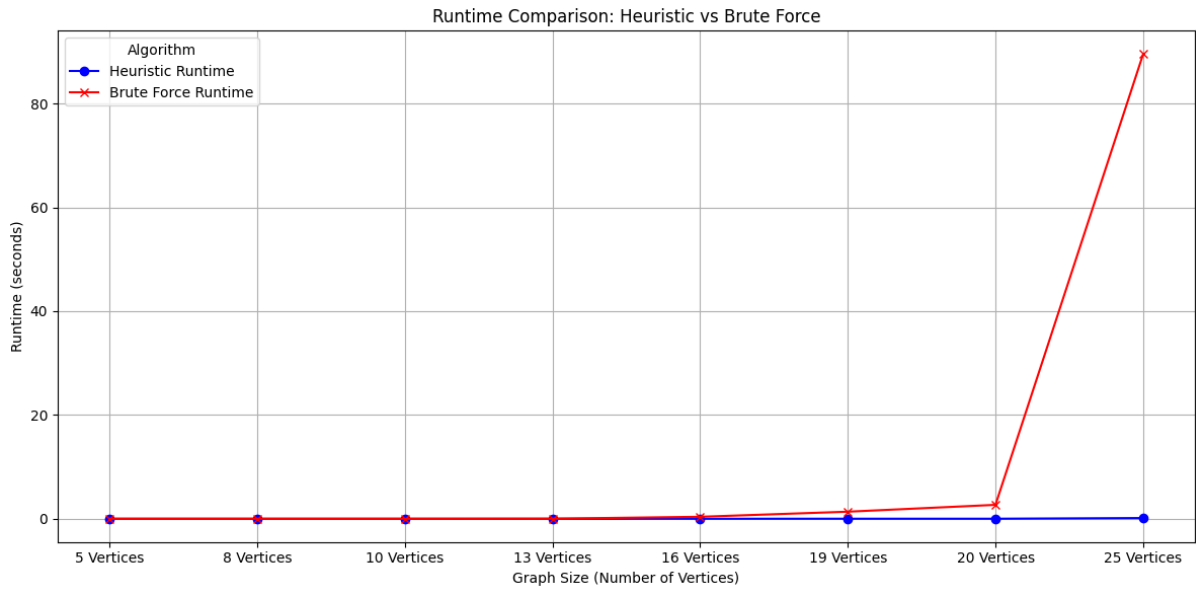


Figure 7: A plot showing the differences of running times of the brute force and heuristic algorithms

By analyzing Figure 7 it can be seen that for the small graph sizes until 19 the running times of both algorithms are similar. However, after that point, the difference starts to become more noticeable and with the graph size 25 the difference of the time complexity is much more visible. Therefore, for the larger problem sizes it is clear that the heuristic would have much less running times.

7.3 Conclusion

Finally, after analyzing the success rate and running time comparisons for both algorithms, it can be concluded that the heuristic algorithm is a practical alternative to the brute force method for larger graph sizes. While the brute force algorithm guarantees exact solutions, it becomes computationally infeasible due to its exponential complexity. By sacrificing a slight bit of success rate—maintaining near-optimal performance above 97%—the heuristic algorithm offers a scalable and efficient solution even for larger problems.

8 Experimental Analysis of the Correctness of the Implementation

Errors may arise in any algorithm due to coding mistakes or logical flaws. To evaluate the correctness of the `find_max_independent_set_heuristic` implementation, a **black-box test** was designed. This approach tests the algorithm's outputs against a set of predetermined cases without diving into the internal workings of the code.

8.1 Test Design

The test was designed to cover a wide range of graph structures to make sure that the algorithm is correct and handles various cases effectively. The test cases include:

1. **Empty Graph:** Graph with no nodes or edges.
Expected behavior: Independent set size is 0.
2. **Single Node:** A graph containing a single node with no edges.
Expected behavior: Independent set contains the single node.
3. **Two Nodes, No Edge:** A graph with two unconnected nodes.
Expected behavior: Independent set contains both nodes.
4. **Complete Graph (K_4):** A fully connected graph with 4 nodes.
Expected behavior: Independent set contains only one node.
5. **Bipartite Graph ($K_{3,4}$):** A bipartite graph with 3 and 4 nodes in each partition.
Expected behavior: Independent set includes all nodes in one partition.
6. **Disconnected Components:** A graph with two disconnected edges.
Expected behavior: Independent set includes one node from each edge.
7. **Cyclic Graph (C_5):** A cycle graph with 5 nodes.
Expected behavior: Independent set includes alternate nodes in the cycle.
8. **Sparse Large Graph:** A random graph with 1000 nodes and 50 edges (very sparse).
Expected behavior: Independent set size is large due to sparsity.
9. **Dense Large Graph:** A random graph with 100 nodes and 4500 edges (very dense).
Expected behavior: Independent set size is small due to high connectivity.

8.2 Test Results

Test Name	Independent Set	Size	Time (s)
Empty Graph	$\{\}$	0	0.000000
Single Node	$\{1\}$	1	0.000000
Two Nodes, No Edge	$\{1, 2\}$	2	0.000000
Complete Graph K_4	$\{0\}$	1	0.000000
Bipartite Graph	$\{3, 4, 5, 6\}$	4	0.001013
Disconnected Components	$\{2, 4\}$	2	0.000000
Cyclic Graph C_5	$\{0, 3\}$	2	0.000000
Sparse Large Graph	$\{\dots\}$ (truncated)	956	19.732920
Dense Large Graph	$\{1, 11, 20, 95\}$	4	0.814001

Table 6: Results of the heuristic algorithm across the black-box test cases

8.3 Analysis of Black-Box Testing

Black-box testing was chosen because:

- It focuses on validating outputs against expected results without considering the algorithm’s internal implementation.
- It allows for the testing of the algorithm under various input conditions, ensuring robust performance and identifying edge cases.

The results demonstrate the following:

- **Correctness for Simple Cases:** For small and simple cases, such as the empty graph, single node, and disconnected components, the heuristic produces the expected results, validating correctness.
- **Robustness for Large Graphs:** For larger and more complex graphs, such as sparse and dense graphs, the heuristic produces results that align with expectations. While the runtime increases significantly for the sparse large graph, the independent set size reflects the quality of the approximation of the heuristic.

8.4 Conclusion

The results confirm that the heuristic algorithm handles all tested cases correctly and does not have any coding or logical errors in the implementation. The algorithm successfully produces optimal results for smaller and simpler graphs and near-optimal results for larger graphs.

9 Discussion

The performance and correctness of the `find_max_independent_set_heuristic` algorithm were evaluated through theoretical and experimental test. This section discusses the results in detail, Checking whether the algorithm has any potential defects or whether there are any inconsistencies between the theoretical and experimental findings.

9.1 Algorithm Performance and Quality

The experimental results show that the heuristic algorithm achieves high-quality solutions for a variety of graph structures. Specifically:

- For small and simple graphs, such as the empty graph, single node, and disconnected components, the algorithm produces optimal results that are close with the theoretical expectations.
- For larger and more complex graphs, such as sparse and dense large graphs, the algorithm maintains a high success rate, achieving near-optimal solutions.

The heuristic algorithm consistently delivers results with a high success rate, as indicated by the success rate plot. This demonstrates the practicality of the algorithm for larger graphs, where brute force approaches become infeasible.

9.2 Correctness Testing and Defects

The correctness tests revealed that the algorithm performs as expected across all test cases. Simple cases, such as the empty graph and single node, produced exact results. However, the heuristic nature of the algorithm means that:

- For larger and denser graphs, there is no guarantee that the algorithm finds the optimal solution.
- The slight deviations in solution quality for larger graphs (e.g., dense large graph) are a usual limitation of heuristic algorithms and not a defect coming from the implementation itself.

No coding defects or logical errors were observed in the correctness tests, as the algorithm consistently produced valid independent sets in all test cases.

9.3 Consistency Between Theoretical and Experimental Results

The theoretical time complexity of the algorithm was determined to be $\mathcal{O}(n^3)$ in the worst case. The experimental analysis, using log-log plots, yielded time complexities of approximately $\mathcal{O}(n^{2.69})$ and $\mathcal{O}(n^{2.73})$ for 50 and 100 runs, respectively. This slight deviation suggests that; the heuristic algorithm scales better in practice than the theoretical analysis predicts, likely due to the tests not being the worst-case scenarios.

The correctness tests and success rate plot confirm that the algorithm's behavior aligns with theoretical expectations in terms of solution quality, with a slight trade-off between optimality and computational efficiency.

9.4 Limitations and Future Improvements

While the heuristic algorithm demonstrates strong performance, the following limitations were identified:

- **Lack of Optimality Guarantees:** Even though the algorithm has a high success rate for most of the graph sizes, it does not guarantee an optimal solution.
- **Slowness:** The algorithm is faster than the brute force since it is in polynomial time, however it is slower than some heuristic algorithms since it tries to balance quality and efficiency. This leads to results with a higher success rate but also higher running times.

Future improvements could include:

- Incorporating additional heuristics to enhance the quality of solutions for larger and denser graphs. Or enhancing the speed by sacrificing a little bit of the quality.

9.5 Conclusion

Overall, the experimental results align closely with theoretical expectations, confirming the correctness and practicality of the algorithm. By sacrificing a slight degree of solution quality, the heuristic algorithm achieves significantly improved computational efficiency, making it a better choice for larger and more complex graphs. The observed results highlight the quality and scalability of the algorithm.

10 References

- Eddy, Duncan, and Mykel J. Kochenderfer. “A Maximum Independent Set Method for Scheduling Earth-Observing Satellite Constellations.” *Journal of Spacecraft and Rockets* 58, no. 5 (September 2021): 1416–29. <https://doi.org/10.2514/1.a34931>.
- Gardiner, Eleanor J, Peter J Artymiuk, and Peter Willett. “Clique-Detection Algorithms for Matching Three-Dimensional Molecular Structures.” *Journal of Molecular Graphics and Modelling* 15, no. 4 (August 1997): 245–53. [https://doi.org/10.1016/s1093-3263\(97\)00089-2](https://doi.org/10.1016/s1093-3263(97)00089-2).
- Khatami, Mostafa, and Amir Salehipour. “The Gradual Minimal Covering Location Problem.” *SSRN Electronic Journal* (2020). <https://doi.org/10.2139/ssrn.3522777>.
- Nieberg, Tim. Independent and dominating sets in wireless communication graphs. *IEEE Transactions on Software Engineering - TSE* (2006). <https://www.researchgate.net/publication/241875153>
- Sharp, Alexa. Proving a Problem is NP-complete. Accessed November 11, 2024. <https://www.cs.cornell.edu/courses/cs482/2005su/handouts/NPComplete.pdf>.
- Ugurlu, Onur. “A New Heuristic Algorithm to Solve the Maximum Independent Set Problem.” *Mathematical and Computational Applications* 18, no. 3 (December 1, 2013): 495–501. <https://doi.org/10.3390/mca18030495>.